Kernel Hacking 2 - Intermedio



Indice

- Concetti di base sul funzionamento del kernel
- Struttura modulare del kernel
- Initramfs
- Completamento descrizione dell’avvio del kernel

Kernel Internals in OSes

- Nel libro *Systems Performance*, di Brendan Gregg, studiamo solo fino alla sezione 3.2.8 del capitolo *Kernel Internals in Operating Systems*: https://www.informit.com/articles/article.aspx?p=3104531
- Se non trovate più queste sezioni online, contattate il docente
- Completeremo lo studio della sezione 3.2 in una delle ultime lezioni

Esecuzione softirq

- Con **`PREEMPT_RT`** disabilitata
  - Equivalenti ad hardirq (non possono dormire, **`in_interrupt == true`**)
    - Ma eseguiti con interruzioni abilitate
    - Possono essere preemptati da hardirq
  - Eseguiti in coda ad hardirq, o demandati al demone (processo) `ksoftirqd` se se ne sono accumulati troppi
- Con **`PREEMPT_RT`** abilitata
  - Eseguiti come thread

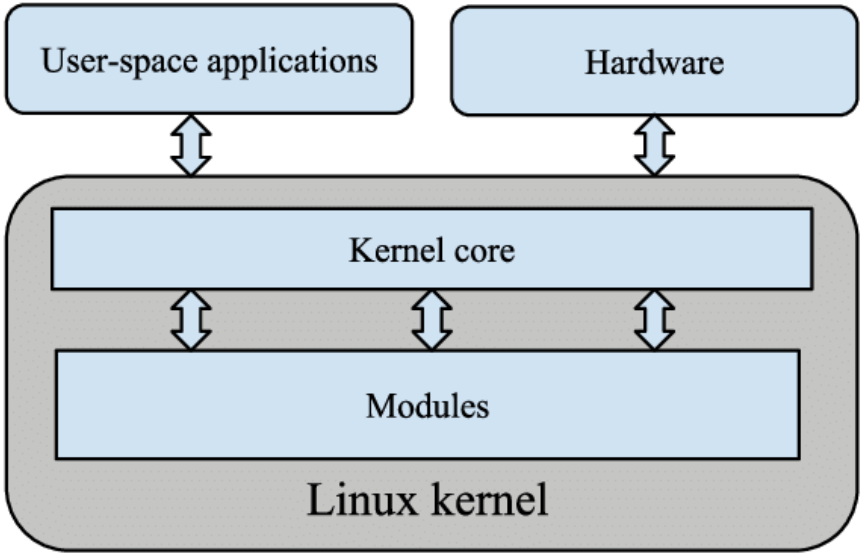
Indice

- Concetti di base sul funzionamento del kernel
- Struttura modulare del kernel
- Initramfs
- Completamento descrizione dell’avvio del kernel

Struttura modulare

- Il kernel Linux è monolitico in termini di spazio di indirizzamento
- Ma è modulare in termini di codice sorgente e binario
- L’intero codice è diviso in due parti:
  - Un *core* contenente tutte funzionalità imprescindibili
  - Un insieme di moduli, contenente tutte le funzionalità non imprenscindibili
- Ossia: ciascun modulo del kernel contiene un componente senza il quale il kernel può funzionare lo stesso, su almeno una architettura hardware

Struttura modulare



Moduli: driver e non solo ...

- Supporto di componenti hardware
  - Le componenti del kernel che forniscono questa funzionalità vengono chiamate *driver*
- Supporto di politiche di gestione delle risorse
  - Scheduler, controller, …
- Supporto della virtualizzazione

...

Caricamento dinamico dei moduli

- La modularizzazione apre la strada ad una ulteriore forma di flessibilità
  - Il *caricamento dinamico*, ossia a tempo di esecuzione, (del codice binario) di un modulo, solo se e quando necessario
- Questo porta a due vantaggi evidenti

  1)Occupazione di memoria solo se e quando necessario

  2)Riduzione delle dimensioni dell'immagine del kernel
- Più un vantaggio meno ovvio, ma che vedremo nel dettaglio
  - Si può modificare il codice eseguibile del kernel che risiede in uno di tali moduli, senza dover ricompilare, reinstallare e riavviare il kernel. Basta infatti rimuovere, modificare, ricompilare e ricaricare solo il modulo

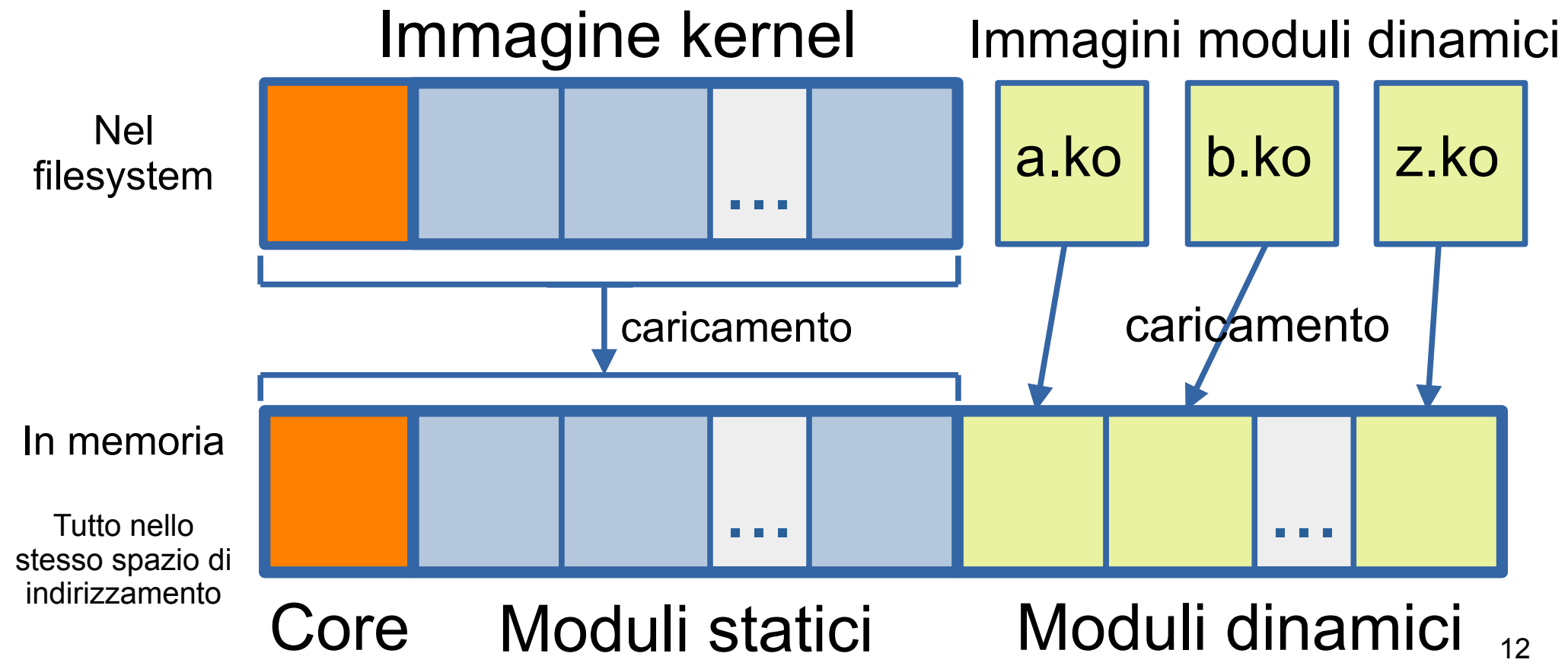
Costo del caricamento dinamico

- Implica l’esecuzione di un meccanismo userspace complesso, per il caricamento del modulo
- Comporta inoltre un problema concettuale irrisolvibile
  - All’avvio del kernel, i moduli da caricare non sarebbero già caricati
  - Sarebbero ovviamente contenuti in un qualche filesystem
  - Se tutti i moduli fossero da caricare, questo includerebbe anche i moduli che contengono i driver per leggere tale filesystem
  - Quindi all’avvio il kernel non sarebbe già in grado di leggere il filesystem
  - Quindi come farebbe il kernel, al suo avvio, a leggere il filesystem in cui sono contenuti proprio i moduli che dovrebbe caricare per poter leggere tale filesystem?

Moduli statici (built-in) e dinamici

- La soluzione ideale è ovviamente avere entrambe le possibilità
  - Moduli statici, cosiddetti *built-in*
    - Vengono collegati staticamente al core del kernel, e quindi fanno parte dell'immagine. Non c'è nulla da caricare
  - Moduli dinamici
    - Oggetti di tipo ELF, che vengono caricati e collegati dinamicamente; e viceversa rimossi dinamicamente
    - Attraverso dei programmi userspace
- Quasi sempre solo i moduli dinamici sono denominati moduli, mentre i moduli built-in sono chiamati semplicemente built-in
- E i moduli dinamici sono chiamati semplicemente moduli, senza l'attributo dinamici. A volte sono anche chiamati moduli caricabili (loadable)

Immagini su fs e in memoria



Modello di memoria

- Sia i moduli statici (che sono già parte dell'immagine) che i moduli dinamici (una volta caricati) risiedono ed accedono allo stesso spazio di indirizzamento del kernel
- Utilizzano (condividono) lo stack unico e lo heap unico del kernel
- Quindi hanno, nello spazio di indirizzamento del kernel: testo e tutte le altri sezioni ELF, stack ed heap
- Nel caso di modulo dinamico
  - L'immagine del modulo è caricata nello spazio di indirizzamento del kernel all'atto appunto del caricamento del modulo
  - I simboli esportati dal modulo sono rilocati, ossia per ogni simbolo viene inizializzata la locazione di memoria ad esso associata, affinché punti alla corretta locazione nello spazio occupato dal memoria nel modulo

Controllo moduli

- Lista dei moduli built-in: contenuta nel file `/lib/modules/<versione-kernel>/modules.builtin`
- Quindi, la lista per il kernel running è nel file `/lib/modules/$(uname -r)/modules.builtin`
- Albero dei moduli dinamici che possono essere caricati, contenuto della directory: `/lib/modules/<versione-kernel>/kernel/`
- Lista dei moduli dinamici correntemente caricati (tra quelli che possono essere caricati): output del comando `lsmod`

Approfondimento moduli

- Nella parte successiva di questo corso approfondiremo l’argomento, e vedremo
  - Caricamento
  - Inizializzazione
  - Rimozione
  - Creazione
  - Modifica configurazione kernel per aggiunta alla compilazione, nel caso di modulo interno
  - Compilazione
  - Installazione

Configurazione tipica

- Tipicamente solo i moduli necessari per l’avvio ed il funzionamento di base del kernel sono compilati built-in
- Tutti gli altri moduli disponibili nel kernel sono compilati come moduli dinamici
- Questo vale anche per
  - I driver necessari per il supporto dei dispositivi di storage, e per i componenti di supporto dei filesystem
  - I driver Human Interface Device (HID): tastiera, mouse, ...
- Perché i possibili dispositivi di storage ed HID sono tantissimi, ed i filesystem sono tanti

Problema filesystem e I/O

- Siccome non ha il supporto per lo storage e per i filsystem built-in, il kernel non può caricare i moduli dinamici all'avvio!
- In particolare non può caricare i moduli che gli servono per caricare i moduli stessi …
- E non può permettere all'utente nessuna interazione in caso di problemi, perché non supporta nessuna interfaccia con l'utente
- Eppure il kernel e lo userspace vengono avviati correttamente ...
- Come sono stati risolti entrambi i problemi?

Indice

- Concetti di base sul funzionamento del kernel
- Struttura modulare del kernel
- Initramfs
- Completamento descrizione dell’avvio del kernel

Initramfs

- Il kernel Linux supporta un tipo di ram filesystem chiamato tmpfs
- Tale supporto non è neanche implementato come modulo, ma sta nel core
- Quindi è sempre presente nell'immagine del kernel
- All'avvio, il kernel crea (ovviamente in ram) un ramfs di tipo tmpfs, tale filesystem è denominato rootfs
- E si aspetta che il bootloader gli fornisca un archivio compresso in formato cpio
- Tale archivio è chiamato initramfs (con grande confusione di termini)
- Il kernel quindi scompatta questo archivio dentro il rootfs
- Per semplicità, chiamiamo initramfs anche il filesystem che a questo punto si ritrova contenuto dentro al rootfs

Contenuto *initramfs*

- L’initramfs contiene
  - Tutti i moduli necessari al kernel per montare il filesystem contenuto nel dispositivo passato come root dalla riga di comando
  - Tutti i moduli necessari per supportare i dispositivi HID
  - Un programma di init, contenuto nella radice dell’albero
    - Carica tutti i moduli presenti nell’initramfs
    - Avvia l’init dello userspace
      - Ossia /sbin/init, se non è stato modificato da riga di comando del kernel (attraverso il l’opzione init=)
- Dettaglio: anche il percorso dell’init nel initramfs può essere modificato, attraverso l’opzione initrd=

Documentazione *initramfs*

- Quasi tutta la documentazione ad oggi disponibile online su initramfs contiene anche informazioni obsolete e scorrette
- Tra la documentazione migliore vi è la seguente
  - https://docs.kernel.org/filesystems/ramfs-rootfs-initramfs.html
    - Concettualmente corretta, scorretta in ogni dettaglio pratico
  - https://wiki.debian.org/initramfs
    - Informazioni tutte corrette, tranne le righe di comando con cpio

Lista e scompattamento

- L’archivio initramfs compresso si trova in `/boot/initrd-<versione_kernel>`
- E’ rimasta la sigla rd nel nome, perché tante versioni fa si tratta di una vera immagine di un ramdisk
- Lista del contenuto dell’initramfs:
  `lsinitramfs <percorso_initramfs>`
- Scompattatamento dell’initramfs:

```
cp <percorso_initramfs> <percorso_vostra_cartella_tmp>
cd <percorso_vostra_cartella_tmp>
unmkinitramfs <nome_file_initramfs> <nome_cartella_in_cui_scompattare>
```

File /init

- Apriamo ed analizziamo il file `/init` nell'*initramfs*
- Contiene, tra i vari comandi
  - `load_modules`
    - Carica i moduli dinamici presenti nell'*initramfs*
  - `mountroot`
    - Monta il filesystem di root
  - `run-init`
    - Invoca l'init nel filesystem di root

Creazione initramfs

- Tramite comando `mkinitramfs`
  - Non vediamo nel dettaglio la sintassi per questo comando
- Crea un *initramfs* per il kernel running, seguendo le istruzioni contenute in un file di configurazione
- Popola l'*initramfs* con un albero di moduli
  - Copiando i moduli da quelli installati per il kernel running
    - Selezionando i moduli indicati nel file di configurazione
- Copia alla radice dell'initramfs il file di init
  - Tipicamente copiato da `/usr/share/initramfs-tools/init`

Indice

- Concetti di base sul funzionamento del kernel
- Struttura modulare del kernel
- Initramfs
- Completamento descrizione dell’avvio del kernel

Completamento avvio kernel

- Torniamo alla `start_kernel`
- Analizziamo la `rest_init`
- L'avvio dell'init userspace è nella `kernel_init`
  - A meno di cambi del parametro `rdinit` da riga comando (o di altri casi che non approfondiamo), questa funzione invoca la `/init` nel ramdisk e ritorna
- L'ultima cosa che fa la `rest_init`, è entrare nell'idle loop, con l'invocazione della `cpu_startup_entry`
  - L'avvio del kernel è finito